

考虑药材内部实际温度场的热湿传递与收缩移动边界模型

摘 要

针对圆柱形药材烘干中“表面先变、内部滞后、长期收缩”的特征，本文在保留题设物性关系的基础上，重新构建一条独立的计算路线：问题二至四不再把药材内部温度直接设为烘房温度，而是同步求解显热传导方程，并将预测温度场反馈给含水率扩散系数。问题一沿用二维轴对称热湿模型作为短时基线，问题四进一步在半径观测驱动的收缩移动域中推进。

数值上采用轴对称有限体积法、后向欧拉时间推进和 Picard 非线性迭代；外边界使用传热、传质串联阻力，Q4 使用保形插值的半径函数和干固体质量守恒。本文计算的主结果为：Q2 在 3 h 时中心和表面温度分别为 49.8504°C 与 49.9670°C，全域最大含水率为 1.76623 kg/kg；Q3 首次低于安全阈值的插值时刻为 207158.1 s，按 60 s 口径报告 207180 s (57.55 h)；Q4 相应为 184080 s (51.13 h)。Q2 的径向剖面显示，温度梯度消失前表面失水明显快于中心，说明内部温度场不能仅作为装饰性变量。

现有附件足以计算“不含潜热的显热温度场—水分扩散”模型，但不足以唯一确定含汽化潜热的模型：尚缺潜热或相变焓、蒸发动力学/平衡关系以及气相传输参数。因而本文把内部温度称为模型预测量，明确给出适用边界，不将其表述为实验真实温度。

关键字： 药材烘干 内部温度场 轴对称有限体积 状态相关扩散 收缩移动边界

1 问题重述与建模目标

药材可近似为长圆柱体。附件 1 给出烘房温度与表面水分边界，附件 2 给出长期半径观测。题目要求求出短时温度/含水率分布、状态相关物性下的含水率变化、固定域达标时间以及考虑收缩后的达标时间。本文将问题二至四的内部温度作为状态量同步求解，并将其反馈给含水率扩散系数。

本文只把“可由已有数据闭合”的显热温度场纳入重新计算，不擅自补写潜热参数。这样既能回答内部实际温度对干燥轨迹的影响，也能清楚说明仍然存在的数据缺口。

2 数据、假设与可计算性审计

2.1 数据与边界

初始温度为 $T_0 = 301.15 \text{ K}$ ，初始干基含水率为 $C_0 = 2.55 \text{ kg/kg}$ ，圆柱长度 $L = 0.25 \text{ m}$ ，初始半径 $R_0 = 0.02 \text{ m}$ 。传热系数取 $h = 25 \text{ W/(m}^2 \text{ K)}$ ，传质系数取 $h_m = 8 \times 10^{-7} \text{ m/s}$ 。附件 1 在 0–4 h 内的环境序列采用分段线性插值；4 h 后采用末一小时平均值恒定延拓。Q4 半径序列采用 PCHIP，超出观测末点后保持末值。

表 1 温度场模型的数据闭合审计

模块	已有数据	状态
显热传导	$T_0, T_\infty(t), \rho(C), c_p(C), k(C), h, L, R(t)$	可计算
水分扩散	$C_0, C_\infty(t), D(C, T), h_m$	可计算
含潜热模型	$L_v(C, T)$ 、相变速率或气液平衡关系	缺失
气相输运	内部蒸汽分压、孔隙率、渗透率等	缺失
实验验证	内部多位置温度/含水率时序	缺失；限制真实性判断

表 1 的审计结果决定了本文只计算无潜热显热温度场；收缩建模所需的质量守恒处理与相关研究中的思路一致 [1]。

2.2 模型假设

药材连续、均质、轴对称，初值均匀；轴线和中截面为对称边界，侧面和端面满足 Robin 交换。所有结果仍不含汽化潜热、吸附热和反应热；因此“实际温度”指由显热方程预测的内部温度，而非实测温度。Q4 采用径向仿射收缩、轴向长度不变、干固体质量守恒。4 h 后的环境延拓是计算假设，不是未来实测数据。

3 热湿耦合模型

3.1 固定域轴对称方程

在半域 $\Omega = \{(r, z) : 0 \leq r \leq R(t), 0 \leq z \leq L/2\}$ 上, 含水率满足

$$\frac{\partial C}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r D(C, T) \frac{\partial C}{\partial r} \right) + \frac{\partial}{\partial z} \left(D(C, T) \frac{\partial C}{\partial z} \right). \quad (1)$$

温度由显热方程给出

$$\rho(C) c_p(C) \frac{\partial T}{\partial t} = \frac{1}{r} \frac{\partial}{\partial r} \left(r k(C) \frac{\partial T}{\partial r} \right) + \frac{\partial}{\partial z} \left(k(C) \frac{\partial T}{\partial z} \right). \quad (2)$$

外边界为

$$-D \nabla C \cdot \mathbf{n} = h_m(C_s - C_\infty), \quad -k \nabla T \cdot \mathbf{n} = h(T_s - T_\infty), \quad (3)$$

轴线和中截面取零法向通量。为识别直接温度假设的影响, 本文把 $T(r, z, t)$ 作为状态量求解, 并在每个非线性迭代中更新 $D(C, T)$; 同时设置 $T = T_\infty(t)$ 的控制组。

Q2、Q3 的物性为

$$\rho = 650 + 128C, \quad c_p = 1450 + \frac{2736C}{C+1}, \quad k = 0.21 + \frac{0.38C}{C+1}, \quad (4)$$

$$D = 2.4 \times 10^{-3} \exp(-0.45/C) \exp(-3850/T), \quad (5)$$

Q4 使用题设给定的另一组关系

$$\rho = 760 + 90C, \quad c_p = 1850 + \frac{2150C}{C+1}, \quad k = 0.12 + \frac{0.20C}{C+1}, \quad (6)$$

$$D = 4.2 \times 10^{-4} \exp(-0.30/C) \exp(-3850/T). \quad (7)$$

温度在指数项中始终使用 K。

3.2 守恒离散与事件判据

对环柱控制体积分, 体积为

$$V_{ij} = \pi(r_{i+1/2}^2 - r_{i-1/2}^2)(z_{j+1/2} - z_{j-1/2}). \quad (8)$$

内部界面采用扩散系数调和平均, 边界把半单元扩散阻力与对流阻力串联。后向欧拉格式的含水率更新写为

$$\frac{V_P}{\Delta t} (C_P^{n+1} - C_P^n) = \sum_N G_{PN} (C_N^{n+1} - C_P^{n+1}) + \sum_b G_b (C_\infty^{n+1} - C_P^{n+1}). \quad (9)$$

温度方程采用同一通量组装，仅将库存项换为 $\rho c_p V$ 、扩散系数换为 k 。每步交替进行 Picard 迭代，线性相对残差控制在 5×10^{-11} 以内。Q4 在当前半径下更新控制体几何和干固体密度；其全域达标条件为

$$\max_{\Omega(t)} C < 0.15 - 10^{-6}. \quad (10)$$

以首次越过阈值的相邻时间步做线性插值，再向上取整到 60 s。

4 问题一：短时二维基线

问题一采用题设常量热物性和给定的 $D(C)$ ，分别计算一维径向和二维轴对称场。1800 s 时二维模型中截面轴心温度为 33.5763°C ，表面为 36.7859°C ，平均含水率为 2.2750 kg/kg ；一维模型平均含水率为 2.2936 kg/kg 。端面提供额外传质通道，使二维平均值降低。图 1 为中截面场量，图 2 为端面效应对照。

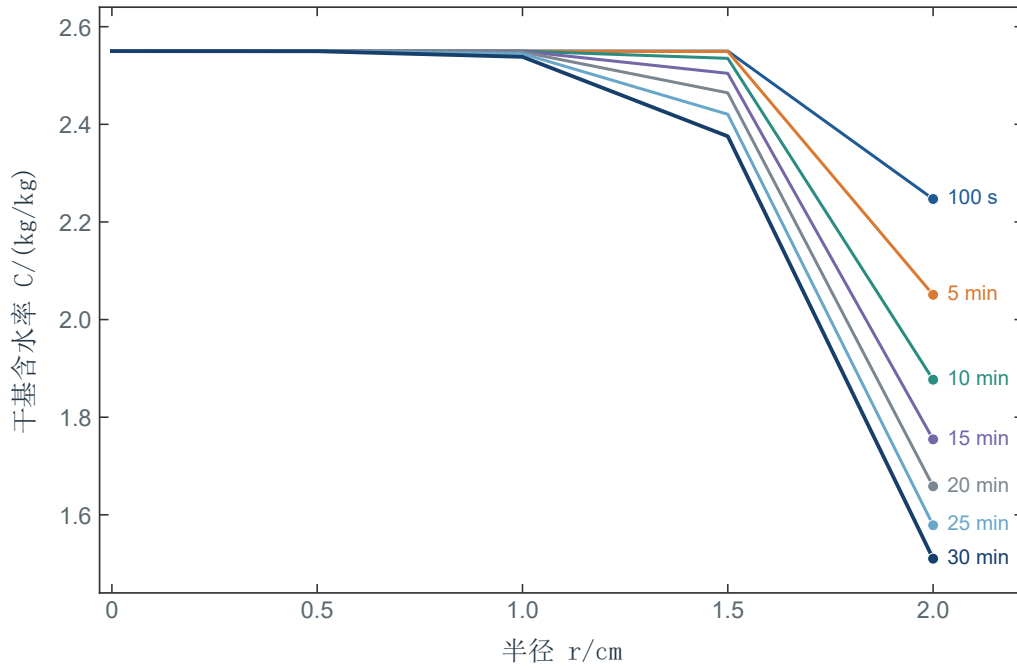


图 1 问题一中截面含水率场

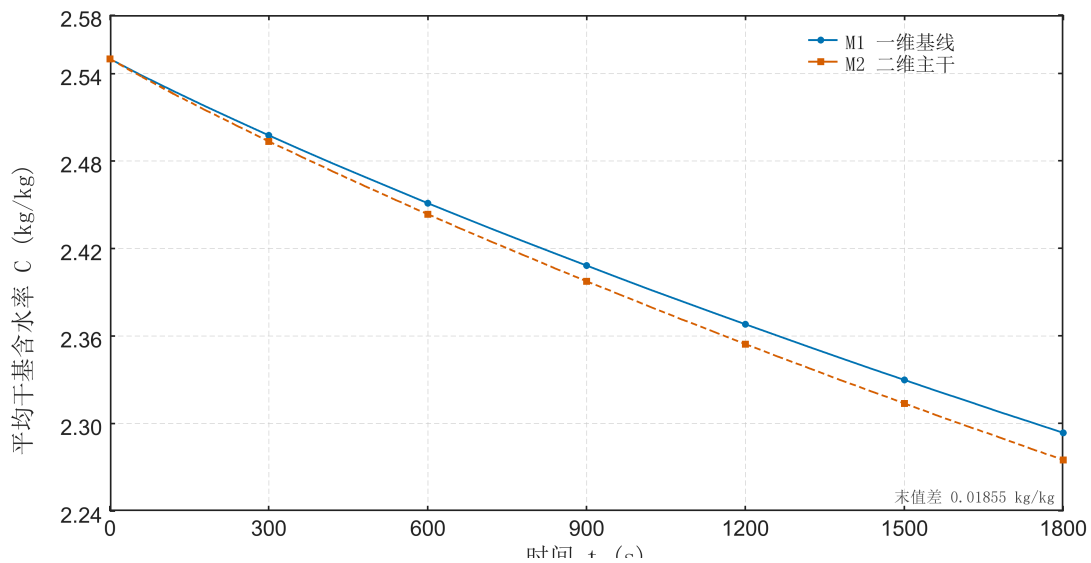


图 2 问题一端面效应对照

5 问题二：内部温度场下的 3 h 过程

5.1 温度场结果

采用 140×36 个控制体、 $\Delta t = 5$ s，从均匀初值重新积分至 3 h。图 3 给出了中截面轴心与表面温度。表面先升温，中心存在明显热滞后；3 h 时差值约 0.1166°C ，温度差虽已变小，但早期差异会通过式 (5) 影响扩散系数。

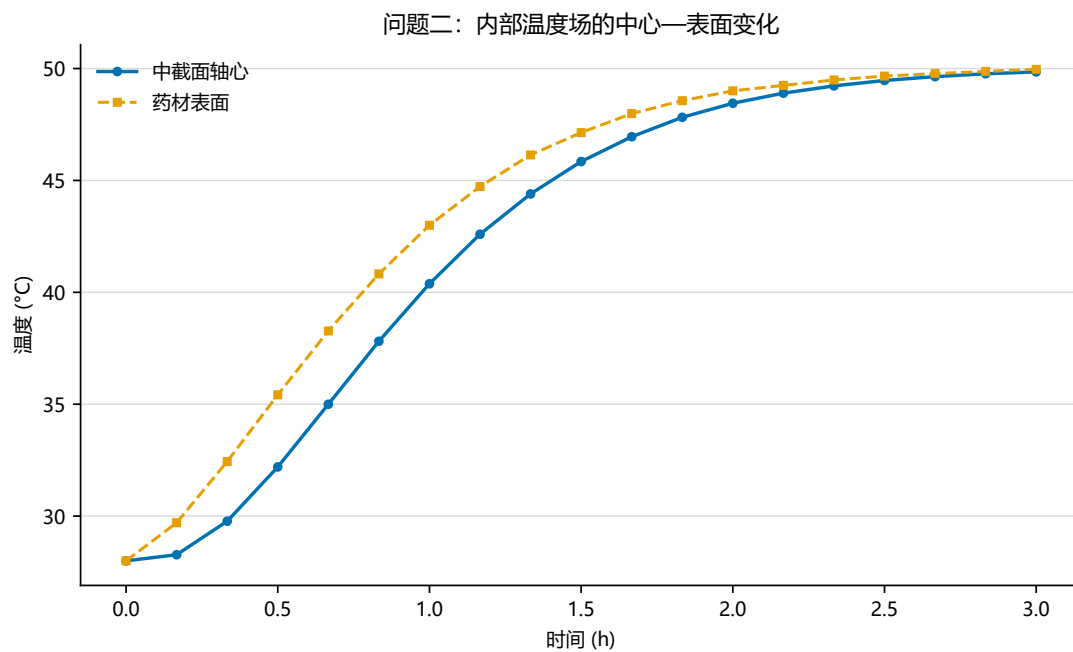


图 3 问题二内部温度场的中心—表面轨迹

表 2 问题二内部温度场的代表性输出

时间/h	中心温度/°C	表面温度/°C	全域最大 C	体积平均 C
0.5	32.1968	35.4177	2.5499	2.2632
1.0	40.3819	43.0000	2.5254	2.0322
1.5	45.8442	47.1379	2.3857	1.8264
2.0	48.4491	49.0025	2.1707	1.6413
2.5	49.4670	49.6624	1.9566	1.4755
3.0	49.8504	49.9670	1.7662	1.3277

表 2 中的中心—表面温差直接来自热方程的解，而不是将环境温度复制到全部单元；这也是传热与传质耦合研究中需要分别检查内部场和边界交换的原因 [2, 3]。

5.2 含水率剖面与路线对照

图 4 在六个时刻给出中截面径向剖面。早期表层梯度最大，随后梯度向中心推进；因此只报告表面含水率会高估全域干燥程度。图 5 将旧的“烘房温度”控制组与本文内部温度场路线并列。三条曲线的网格和时间步不完全相同，图中只作机制对照，不作精度排序。

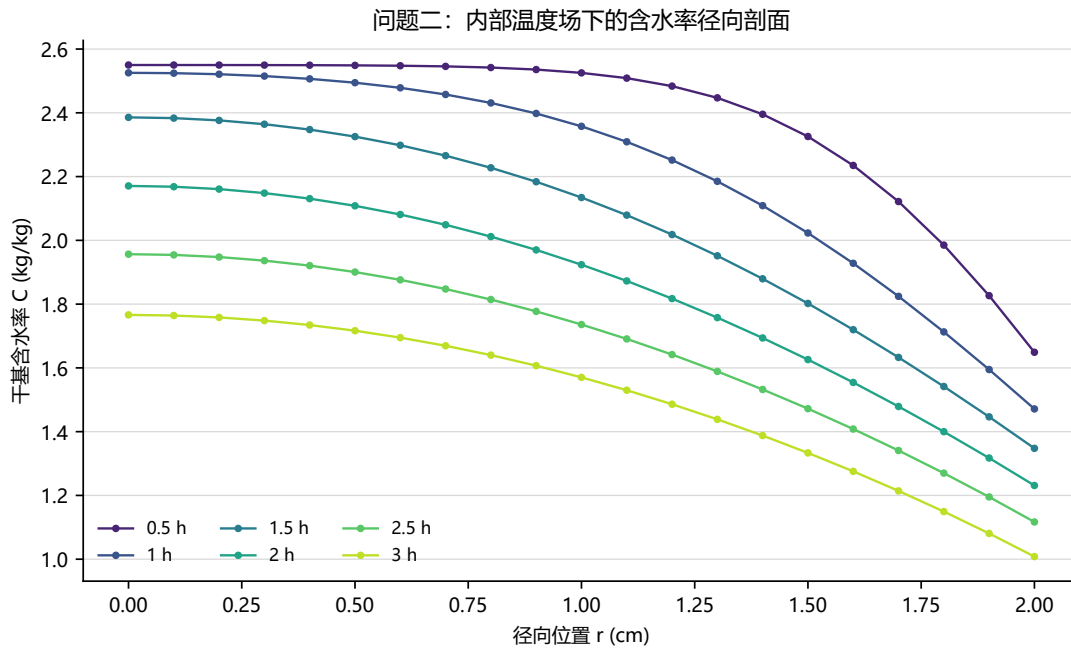


图 4 问题二内部温度场下的含水率径向剖面

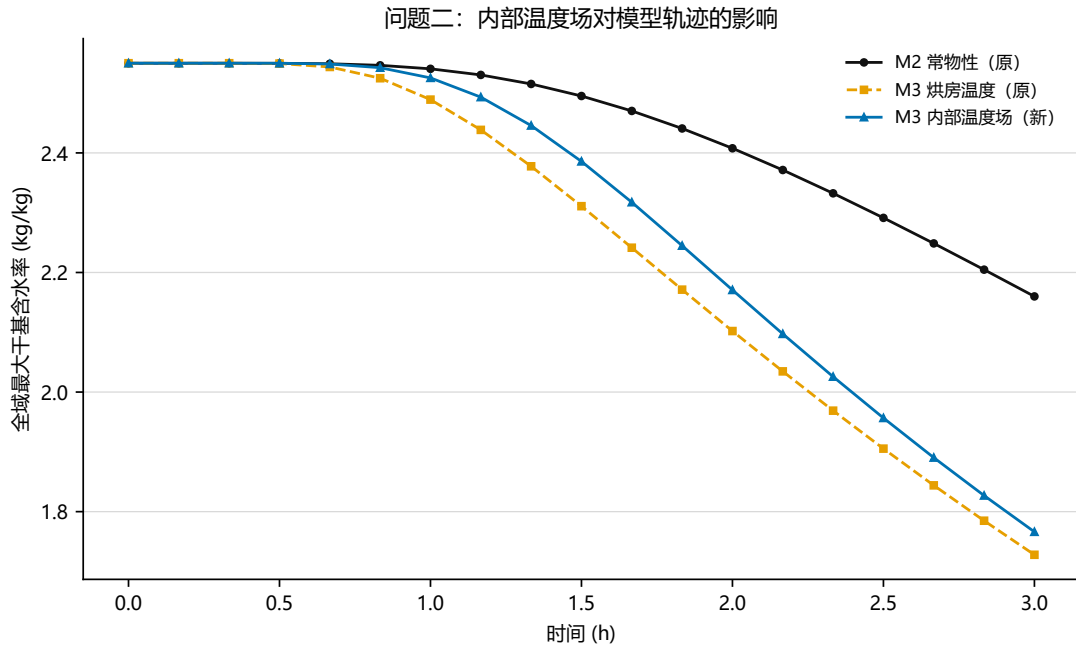


图 5 内部温度场路线与旧控制路线的轨迹对照

6 问题三：固定域全域达标时间

问题三从 $t = 0$ 重新积分。采用 140×36 网格和 $\Delta t = 60$ s，4 h 后按末段平均环境延拓。首次越过式 (10) 的插值时刻为 207158.1 s，按 60 s 口径报告为 207180 s，即 57.55 h。末步的中心和表面温度均为 49.9989°C ，说明到长期达标时热滞后已基本消失，但它仍改变了早期扩散历史。图 6 给出轨迹与插值事件。

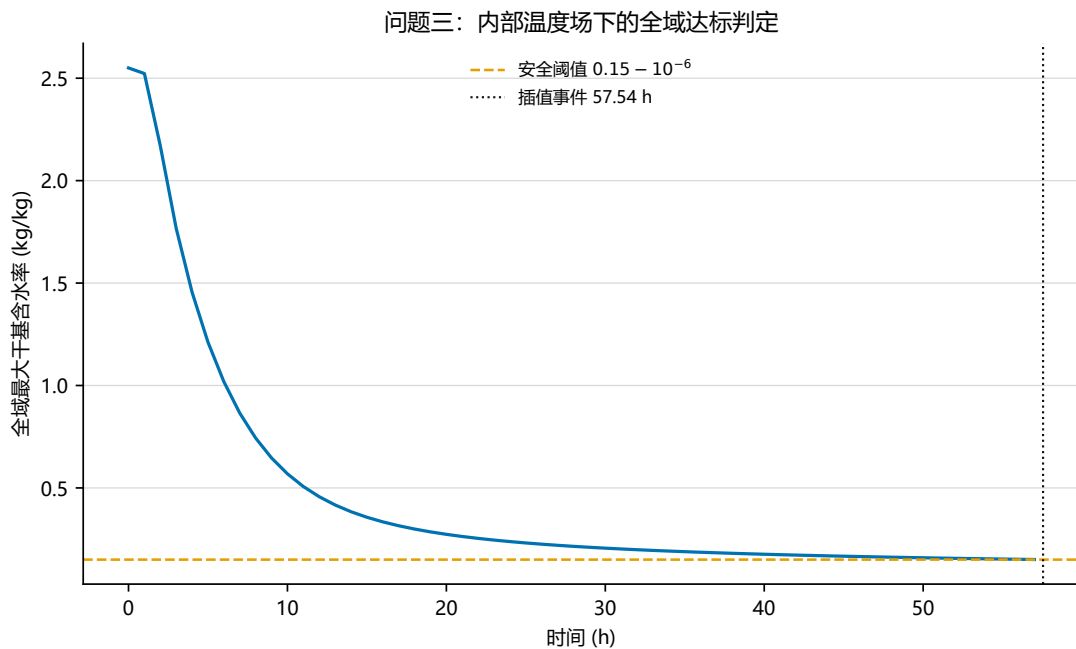


图 6 问题三内部温度场下的全域达标判定

7 问题四：收缩移动域下的达标时间

Q4 使用附件半径序列构造 $R(t)$ ，在每个时间步更新移动控制体、干固体密度和 Q4 物性。图 7 显示半径从 2.00 cm 向 1.20 cm 收缩；这既缩短扩散距离，也改变边界面积和库存权重，不能只用距离平方比例修正。

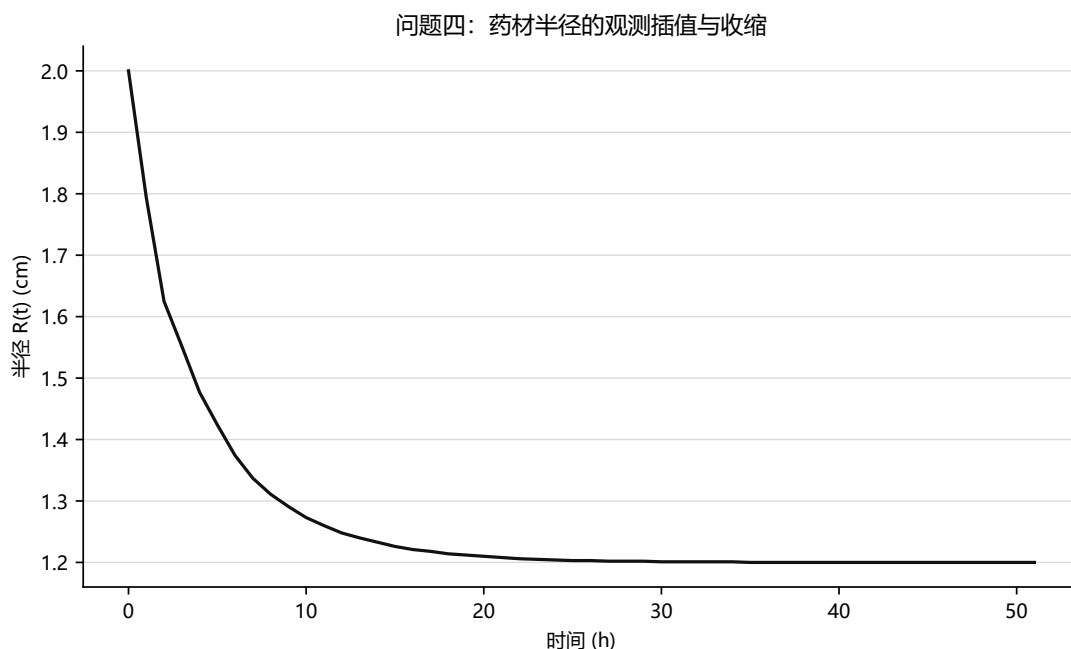


图 7 问题四半径观测插值与收缩轨迹

首次越过阈值的插值时刻为 184079.97 s，报告时刻为 184080 s，即 51.13 h。图 8 给出全域最大含水率轨迹。该值与问题三的差异同时包含 Q4 物性改变和收缩几何效应，不能解释为纯收缩贡献。

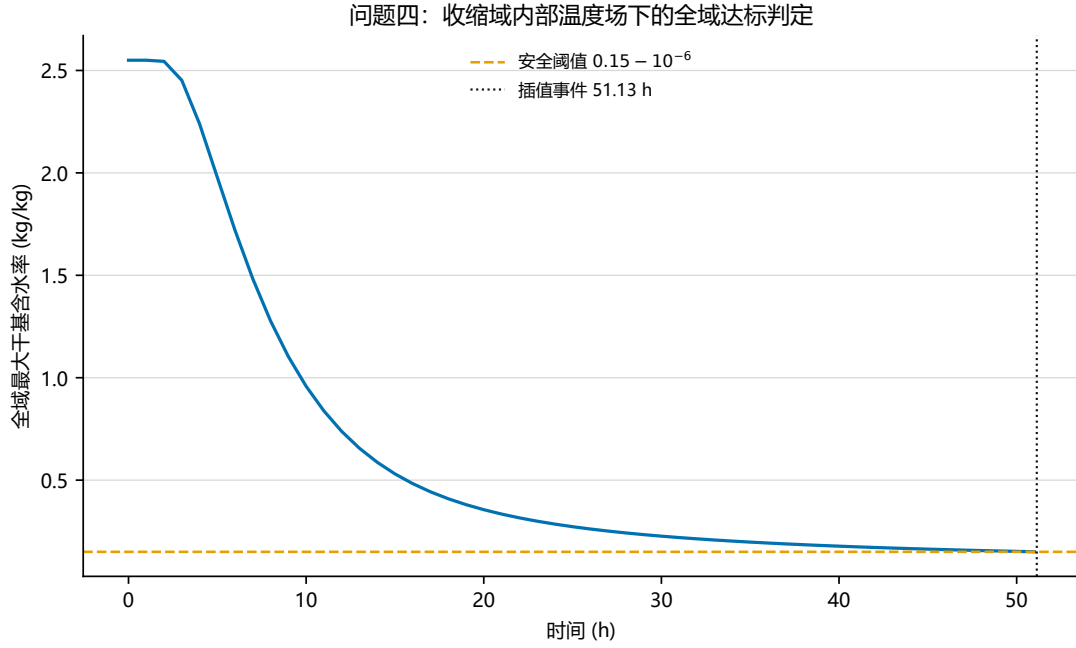


图 8 问题四收缩域内部温度场下的全域达标判定

8 数值检验、局限与结论

8.1 守恒与残差

Q2–Q4 新路线的最大线性相对残差分别为 5.00×10^{-11} 、 5.01×10^{-11} 和 5.01×10^{-11} ；归一化水分平衡误差分别为 2.02×10^{-8} 、 1.67×10^{-9} 和 1.22×10^{-9} 。这些数值检验说明离散方程求解充分、库存与边界通量基本闭合，但不等于实验验证，也不能消除时间步和空间网格误差。

表 3 计算的数值质量指标

算例	网格（径向 × 轴向）	时间步/s	最大平衡误差
Q2 内部温度场	140 × 36	5	2.02×10^{-8}
Q3 固定域	140 × 36	60	1.67×10^{-9}
Q4 收缩移动域	140 × 32	60	1.22×10^{-9}

表 3 给出可复现实验所需的网格、步长和守恒误差，结果文件中的线性残差还可由日志逐步核对。

8.2 局限与缺失数据

本文未加入潜热，因为附件没有 $L_v(C, T)$ 、相变速率/气液平衡关系，也没有内部蒸汽分压、孔隙率和渗透率等气相传输参数。若强行加入潜热，只能任意补参数，无法得到唯一、可审计的答案。内部多点温度和含水率观测同样缺失，因此当前温度是模型预

测而不是实验事实。后续若补充这些数据，应先进行潜热参数辨识和独立批次验证，再把含潜热路线升级为主模型。

8.3 结论

在现有数据可闭合的范围内，本文完成了“显热内部温度场—状态相关水分扩散—收缩移动域”的计算。Q2 的 3 h 中心/表面温度为 49.8504/49.9670°C，最大含水率为 1.76623 kg/kg；Q3、Q4 的 60 s 基准报告时间分别为 57.55 h 和 51.13 h。本文明确呈现了升温阶段的内部热滞后，并将潜热数据缺口列为后续验证工作。所有结果均附带网格、时间步、边界延拓和“无潜热”限定。

参考文献

- [1] Mahiuddin M, Khan M I H, Kumar C, et al. Shrinkage of Food Materials During Drying: Current Status and Challenges[J]. Comprehensive Reviews in Food Science and Food Safety, 2018, 17(5): 1113–1126.
- [2] Defraeye T, Blocken B, Carmeliet J. Analysis of convective heat and mass transfer coefficients for convective drying of a porous flat plate by conjugate modelling[J]. International Journal of Heat and Mass Transfer, 2012, 55: 112–124.
- [3] Khan M I H, Batuwatta-Gamage C P, Karim M A, et al. Fundamental Understanding of Heat and Mass Transfer Processes for Physics-Informed Machine Learning-Based Drying Modelling[J]. Energies, 2022, 15: 9347.

AI 工具使用声明

本参赛队使用 AI 工具辅助代码调试、数值计算、图表制作和文字整理；模型假设、数据审计、结果复核及最终提交由参赛队负责。论文源文件、运行日志和支撑材料均按提交包目录保存。

A 计算与图件文件

核心程序、运行脚本和图件脚本位于 `code-sources/`。结果 CSV 位于 `results/`，包含温度、含水率轨迹及径向剖面。计算清单 `revision-manifest.json` 记录源程序哈希、数据审计和缺失字段。

B 核心程序（节选）

以下为本文实际执行的显热温度场求解器、批量运行程序、结果固化程序和图件脚本。它们与正文中的方程、参数和结果一一对应，便于评委复核。

B.1 温度场求解器

```
"""Formal COMPUTE-stage solver for CUMCM 2026 problem A.
```

```
The implementation follows the frozen H2 route: M1 is the 1-D radial
baseline, M2 is the 2-D axisymmetric constant-law backbone, M3 adds the
problem-supplied state laws, and M4 is a controlled temperature-field
ablation. Only files below 04-compute are written.
```

```
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import csv
```

```
import hashlib
```

```
import json
```

```
import math
```

```
import platform
```

```
import sys
```

```
import time
```

```
from dataclasses import dataclass
```

```
from datetime import datetime, timezone
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import openpyxl
```

```
from PIL import Image, ImageDraw, ImageFont
```

```

ROOT = Path(__file__).resolve().parents[2]
OUT = ROOT / "04-compute"
RESULTS = OUT / "results"
FIGURES = OUT / "diagnostic-figures"
LOGS = OUT / "logs"
R0 = 0.02
LENGTH = 0.25
SEED = 20260911
EVAL_R = np.linspace(0.0, R0, 21)
PICARD_TOL = 1e-7

def sha256(path: Path) -> str:
    h = hashlib.sha256()
    with path.open("rb") as f:
        for chunk in iter(lambda: f.read(1024 * 1024), b''):
            h.update(chunk)
    return h.hexdigest()

def locate_xlsx(size: int) -> Path:
    matches = [p for p in (ROOT / "input").rglob("*.xlsx") if p.stat().st_size == size]
    if len(matches) != 1:
        raise RuntimeError(f"expected one workbook of {size} bytes, found {len(matches)}")
    return matches[0]

def load_inputs():
    p1, p2 = locate_xlsx(16586), locate_xlsx(11485)
    wb1 = openpyxl.load_workbook(p1, read_only=True, data_only=True)
    ws1 = wb1[wb1.sheetnames[0]]
    env = np.asarray(list(ws1.iter_rows(min_row=2, values_only=True)), dtype=float)
    wb2 = openpyxl.load_workbook(p2, read_only=True, data_only=True)
    ws2 = wb2[wb2.sheetnames[0]]

```

```

radius = np.asarray(list(ws2.iter_rows(min_row=2, values_only=True)), dtype=float)
if env.shape != (241, 3) or radius.shape != (145, 2):
    raise RuntimeError(f"input shape audit failed: {env.shape}, {radius.shape}")
if not (env[0, 0] == 0 and env[-1, 0] == 14400 and radius[0, 0] == 0 and radius[-1, 0] == 14400):
    raise RuntimeError("input endpoint audit failed")
if not np.all(np.diff(radius[:, 1]) <= 1e-12):
    raise RuntimeError("radius is not monotone non-increasing")
return p1, env, p2, radius

P1, ENV, P2, RADIUS_DATA = load_inputs()

def terminal_value(col: int, window_s: int, statistic: str = "mean") -> float:
    mask = ENV[:, 0] >= ENV[-1, 0] - window_s
    vals = ENV[mask, col]
    return float(np.median(vals) if statistic == "median" else np.mean(vals))

def outside(t: float, window_s: int = 3600) -> tuple[float, float]:
    if t <= ENV[-1, 0]:
        return (float(np.interp(t, ENV[:, 0], ENV[:, 1])) + 273.15,
                float(np.interp(t, ENV[:, 0], ENV[:, 2])))
    return terminal_value(1, window_s) + 273.15, terminal_value(2, window_s)

def pchip_slopes(x: np.ndarray, y: np.ndarray) -> np.ndarray:
    h = np.diff(x); delta = np.diff(y) / h; n = len(x)
    d = np.zeros(n)
    for k in range(1, n - 1):
        if delta[k - 1] * delta[k] > 0:
            w1, w2 = 2 * h[k] + h[k - 1], h[k] + 2 * h[k - 1]
            d[k] = (w1 + w2) / (w1 / delta[k - 1] + w2 / delta[k])
    def endpoint(h0, h1, del0, del1):
        val = ((2 * h0 + h1) * del0 - h0 * del1) / (h0 + h1)
        if val * del0 <= 0: return 0.0

```

```

        if del0 * del1 < 0 and abs(val) > abs(3 * del0): return 3 * del0
    return val
d[0] = endpoint(h[0], h[1], delta[0], delta[1])
d[-1] = endpoint(h[-1], h[-2], delta[-1], delta[-2])
return d

R_TIME = RADIUS_DATA[:, 0]
R_VALUES = RADIUS_DATA[:, 1] / 100.0
R_SLOPES = pchip_slopes(R_TIME, R_VALUES)

def radius_at(t: float, mode: str = "pchip") -> float:
    if mode == "constant": return R0
    if t <= R_TIME[0]: return float(R_VALUES[0])
    if t >= R_TIME[-1]: return float(R_VALUES[-1])
    if mode == "linear": return float(np.interp(t, R_TIME, R_VALUES))
    i = int(np.searchsorted(R_TIME, t) - 1)
    h = R_TIME[i + 1] - R_TIME[i]; s = (t - R_TIME[i]) / h
    h00 = 2*s**3 - 3*s**2 + 1; h10 = s**3 - 2*s**2 + s
    h01 = -2*s**3 + 3*s**2; h11 = s**3 - s**2
    return float(h00*R_VALUES[i] + h10*h*R_SLOPES[i] + h01*R_VALUES[i+1] + h11*h*R_SL

@dataclass
class Grid:
    nr: int
    nz: int
    radius: float = R0
    stretch: float = 1.7
    radial_strategy: str = "two-sided-local"

    def __post_init__(self):
        s = np.linspace(0, 1, self.nr + 1)
        # Smooth two-sided radial refinement: resolve both the symmetry-axis
        # maximum and the outer Robin boundary layer without zero-width cells.

```

```

if self.radial_strategy == "uniform":
    self.rf = self.radius * s
elif self.radial_strategy == "two-sided-local":
    self.rf = self.radius * (s - 0.65 * np.sin(2 * np.pi * s) / (2 * np.pi))
else:
    raise ValueError(f"unknown radial strategy: {self.radial_strategy}")
self.rc = (2.0/3.0) * (self.rf[1:]**3-self.rf[:-1]**3) / np.maximum(self.rf[1:]
if self.nz == 1:
    self.zf = np.array([0.0, LENGTH]); self.zc = np.array([LENGTH / 2])
else:
    q = np.linspace(0, 1, self.nz + 1)
    self.zf = LENGTH * 0.5 * (1 - np.cos(np.pi * q))
    self.zc = 0.5 * (self.zf[:-1] + self.zf[1:])
self.dr = np.diff(self.rf); self.dz = np.diff(self.zf)
self.annulus = math.pi * np.diff(self.rf ** 2)
self.half_domain = self.nz > 1
self.axial_extent = LENGTH / 2 if self.half_domain else LENGTH
if self.half_domain:
    q = np.linspace(0, 1, self.nz + 1)
    self.zf = self.axial_extent * (1 - (1 - q) ** self.stretch)
    self.zc = 0.5 * (self.zf[:-1] + self.zf[1:])
    self.dz = np.diff(self.zf)
self.vol = self.annulus[:, None] * self.dz[None, :]

def harmonic(a, b):
    return 2 * a * b / np.maximum(a + b, 1e-300)

def conductances(coeff, grid: Grid, conv: float, include_ends: bool = True):
    gr = harmonic(coeff[:-1, :], coeff[1:, :]) * (
        2 * math.pi * grid.rf[1:-1, None] * grid.dz[None, :]
    ) / (grid.rc[1:, None] - grid.rc[:-1, None])
    if grid.nz > 1:
        gz = harmonic(coeff[:, :-1], coeff[:, 1:]) * grid.annulus[:, None] / (
            grid.zc[None, 1:] - grid.zc[None, :-1]

```

```

    )
else:
    gz = np.zeros((grid.nr, 0))
ar = 2 * math.pi * grid.radius * grid.dz
gbr = ar / ((grid.radius-grid.rc[-1]) / np.maximum(coeff[-1, :], 1e-300) + 1 / co
if include_ends and grid.nz > 1:
    gb0 = np.zeros(grid.nr) if grid.half_domain else grid.annulus / (0.5 * grid.d
    gb1 = grid.annulus / (0.5 * grid.dz[-1] / np.maximum(coeff[:, -1], 1e-300) +
else:
    gb0 = np.zeros(grid.nr); gb1 = np.zeros(grid.nr)
return gr, gz, gbr, gb0, gb1

def implicit_linear(old, coeff, storage, grid: Grid, dt, conv, env_value, include_end
    x0=None, rtol=5e-11, maxiter=500):
gr, gz, gbr, gb0, gb1 = conductances(coeff, grid, conv, include_ends)
time_diag = storage * grid.vol / dt
diag = time_diag.copy()
diag[:-1, :] += gr; diag[1:, :] += gr
if grid.nz > 1:
    diag[:, :-1] += gz; diag[:, 1:] += gz
diag[-1, :] += gbr; diag[:, 0] += gb0; diag[:, -1] += gb1
rhs = time_diag * old
rhs[-1, :] += gbr * env_value
rhs[:, 0] += gb0 * env_value; rhs[:, -1] += gb1 * env_value

def amat(x):
    y = time_diag * x
    q = gr * (x[:-1, :] - x[1:, :]); y[:-1, :] += q; y[1:, :] -= q
    if grid.nz > 1:
        qz = gz * (x[:, :-1] - x[:, 1:]); y[:, :-1] += qz; y[:, 1:] -= qz
    y[-1, :] += gbr * x[-1, :]
    y[:, 0] += gb0 * x[:, 0]; y[:, -1] += gb1 * x[:, -1]
    return y

x = old.copy() if x0 is None else x0.copy()

```

```

r = rhs - amat(x); z = r / np.maximum(diag, 1e-300); p = z.copy()
rz = float(np.sum(r * z)); rhs_norm = max(float(np.linalg.norm(rhs.ravel())), 1e-
it = 0
for it in range(1, maxiter + 1):
    ap = amat(p); denom = float(np.sum(p * ap))
    if denom <= 0: raise RuntimeError("non-SPD operator in CG")
    alpha = rz / denom; x += alpha * p; r -= alpha * ap
    if float(np.linalg.norm(r.ravel())) <= rtol * rhs_norm: break
    z = r / np.maximum(diag, 1e-300); rz_new = float(np.sum(r * z))
    p = z + (rz_new / rz) * p; rz = rz_new
else:
    raise RuntimeError(f"CG failed after {maxiter} iterations")
boundary_in = float(np.sum(gbr * (env_value - x[-1, :])) +
                    np.sum(gb0 * (env_value - x[:, 0])) +
                    np.sum(gb1 * (env_value - x[:, -1])))
residual = float(np.linalg.norm((amat(x) - rhs).ravel()) / rhs_norm)
return x, boundary_in, it, residual

```

```

def implicit_reference_linear(old, coeff, ref_grid: Grid, physical_grid: Grid, dt, co
    env_value, rho0=820.0, include_ends=True, x0=None,
    rtol=5e-11, maxiter=500):

```

"""Assemble the fixed-reference-domain equation independently.

Storage uses $\rho_0 dV_{\text{ref}}$. Flux conductances are transformed from the current physical faces with $\rho_d = \rho_0 (R_0/R)^2$, which supplies the radial Jacobian/metric factors and current-surface Robin term explicitly.

"""

```

rho_d = rho0 * (R0 / physical_grid.radius) ** 2
gr0, gz0, gbr0, gb00, gb10 = conductances(coeff, physical_grid, conv, include_end
gr, gz, gbr, gb0, gb1 = rho_d*gr0, rho_d*gz0, rho_d*gbr0, rho_d*gb00, rho_d*gb10
time_diag = rho0 * ref_grid.vol / dt
diag = time_diag.copy(); diag[:-1,:] += gr; diag[1:,:] += gr
if ref_grid.nz > 1: diag[:, -1] += gz; diag[:, 1] += gz
diag[-1,:] += gbr; diag[:, 0] += gb0; diag[:, -1] += gb1
rhs = time_diag*old; rhs[-1,:] += gbr*env_value; rhs[:, 0] += gb0*env_value; rhs[:, -1] +=

```

```

def amat(x):
    y=time_diag*x; q=gr*(x[:-1,:]-x[1:,:]); y[:-1,:]+=q; y[1:,:]-=q
    if ref_grid.nz>1:
        qz=gz*(x[:, :-1]-x[:, 1:]); y[:, :-1]+=qz; y[:, 1:]-=qz
    y[-1,:]+=gbr*x[-1,:]; y[:, 0]+=gb0*x[:, 0]; y[:, -1]+=gb1*x[:, -1]; return y
x=old.copy() if x0 is None else x0.copy(); r=rhs-amat(x); z=r/np.maximum(diag,1e-
rz=float(np.sum(r*z)); rhs_norm=max(float(np.linalg.norm(rhs.ravel()))),1e-300); i
for it in range(1,maxiter+1):
    ap=amat(p); denom=float(np.sum(p*ap))
    if denom<=0: raise RuntimeError("non-SPD reference operator")
    alpha=rz/denom; x+=alpha*p; r-=alpha*ap
    if float(np.linalg.norm(r.ravel()))<=rtol*rhs_norm: break
    z=r/np.maximum(diag,1e-300); rz_new=float(np.sum(r*z)); p=z+(rz_new/rz)*p; rz
else: raise RuntimeError(f"reference CG failed after {maxiter} iterations")
boundary_in=float(np.sum(gbr*(env_value-x[-1,:]))+np.sum(gb0*(env_value-x[:, 0]))+
residual=float(np.linalg.norm((amat(x)-rhs).ravel())/rhs_norm)
return x,boundary_in,it,residual

def props_q1(c):
    return (np.full_like(c, 820.0), np.full_like(c, 2600.0),
            np.full_like(c, 0.36), 7e-9 * np.exp(-0.89 / np.maximum(c, 1e-9)))

def props_q23(c, temp_k, constant=False, pref=1.0, exponent=1.0):
    cc = np.full_like(c, 2.55) if constant else c
    tt = np.full_like(temp_k, 301.15) if constant else temp_k
    rho = 650 + 128 * cc; cp = 1450 + 2736 * cc / (cc + 1)
    k = 0.21 + 0.38 * cc / (cc + 1)
    d = 2.4e-3 * pref * np.exp(-0.45 * exponent / np.maximum(cc, 1e-9)) * np.exp(-385
    return rho, cp, k, d

def props_q4(c, temp_k, pref=1.0, exponent=1.0):
    rho = 760 + 90 * c; cp = 1850 + 2150 * c / (c + 1)
    k = 0.12 + 0.20 * c / (c + 1)

```

```

d = 4.2e-4 * pref * np.exp(-0.30 * exponent / np.maximum(c, 1e-9)) * np.exp(-3850
return rho, cp, k, d

def surface_value(cell, coeff, halfwidth, conv, env):
    return (coeff * cell / halfwidth + conv * env) / (coeff / halfwidth + conv)

def axis_mid_value(field, grid: Grid):
    if grid.nz == 1: line=field[:,0]
    else:
        z1,z2=grid.zc[0]**2,grid.zc[1]**2
        line=(field[:,0]*z2-field[:,1]*z1)/(z2-z1)
    if grid.nr < 2: return float(line[0])
    x1,x2=grid.rc[0]**2,grid.rc[1]**2
    return float((line[0]*x2-line[1]*x1)/(x2-x1))

def radial_profile(field, coeff, grid: Grid, conv, env, positions=EVAL_R,
                    return_boundary=False):
    j = 0
    if grid.nz>1:
        z1,z2=grid.zc[0]**2,grid.zc[1]**2
        midline=(field[:,0]*z2-field[:,1]*z1)/(z2-z1)
        coeffline=(coeff[:,0]*z2-coeff[:,1]*z1)/(z2-z1)
    else: midline=field[:,0]; coeffline=coeff[:,0]
    surface = surface_value(float(midline[-1]), float(coeffline[-1]),
                            grid.radius-grid.rc[-1], conv, env)
    xp = np.r_[0.0, grid.rc, grid.radius]
    fp = np.r_[axis_mid_value(field,grid), midline, surface]
    pos = np.asarray(positions)
    vals = np.interp(np.minimum(pos, grid.radius), xp, fp)
    vals[pos > grid.radius + 1e-12] = np.nan
    if return_boundary:
        return vals, float(surface)
    return vals

```

```

@dataclass
class RunConfig:
    label: str
    nr: int
    nz: int
    dt: float
    duration: float
    q: int
    model: str
    h_mult: float = 1.0
    hm_mult: float = 1.0
    pref: float = 1.0
    exponent: float = 1.0
    window_s: int = 3600
    stop_threshold: float | None = None
    radius_mode: str = "pchip"
    moving_impl: str = "fixed"
    sample_every: float = 60.0
    radial_strategy: str = "two-sided-local"
    # ``prescribed`` keeps the frozen route. ``field`` solves the sensible-
    # heat equation and is used only by the isolated temperature-field revision.
    temp_mode: str = "prescribed"

def simulate(cfg: RunConfig, initial=None):
    moving = cfg.q == 4
    radius = radius_at(0, cfg.radius_mode) if moving else R0
    grid = Grid(cfg.nr, cfg.nz, radius, radial_strategy=cfg.radial_strategy)
    ref_grid = Grid(cfg.nr, cfg.nz, R0, radial_strategy=cfg.radial_strategy)
    c = np.full((cfg.nr, cfg.nz), 2.55) if initial is None else initial[0].copy()
    temp = np.full_like(c, 301.15) if initial is None else initial[1].copy()
    initial_inventory = float(np.sum(c * grid.vol))
    cumulative_flux = 0.0; max_cg = 0; max_linres = 0.0; max_picard = 0
    max_balance = 0.0; records = []; states = []; max_picard_update = 0.0

```

```

next_sample = 0.0; start = time.perf_counter(); previous_max = max(float(c.max()))
bracket = None; dry_inventory_rel = 0.0; geom_rel = 0.0; local_dry_residual = 0.0
nsteps = int(math.ceil(cfg.duration / cfg.dt))
for step in range(nsteps + 1):
    t = min(step * cfg.dt, cfg.duration)
    tenv, cenv = outside(t, cfg.window_s)
    current_radius = radius_at(t, cfg.radius_mode) if moving else R0
    if moving and abs(current_radius - grid.radius) > 1e-15:
        old_grid = grid; grid = Grid(cfg.nr, cfg.nz, current_radius, radial_strategy)
        # Material-coordinate cells retain C and T; dry density follows continuity
        geom_rel = max(geom_rel, abs(np.sum(grid.vol) / (math.pi * current_radius**2)))
    if t + 1e-9 >= next_sample or step == nsteps:
        if cfg.q == 1: rho, cp, k, d = props_q1(c)
        elif cfg.q in (2, 3): rho, cp, k, d = props_q23(c, temp, constant=(cfg.moving == 0))
        else: rho, cp, k, d = props_q4(c, temp, pref=cfg.pref, exponent=cfg.exponent)
        rp_c, surface_c_boundary = radial_profile(
            c, d, grid, 8e-7 * cfg.hm_mult, cenv, return_boundary=True)
        rp_t, surface_t_boundary = radial_profile(
            temp, k, grid, 25 * cfg.h_mult, tenv, return_boundary=True)
        rec = {"time_s": t, "radius_m": current_radius, "max_C": max(float(c.max()),
            "min_C": float(c.min()), "mean_C": float(np.sum(c*grid.vol)/np.sum(grid.vol)),
            "center_C": float(rp_c[0]), "surface_C": float(surface_c_boundary),
            "center_T_C": float(rp_t[0] - 273.15), "surface_T_C": float(surface_t_boundary - 273.15)}
        records.append(rec); states.append((t, rp_t - 273.15, rp_c))
        next_sample += cfg.sample_every
    current_max = max(float(c.max()), axis_mid_value(c, grid))
    if cfg.stop_threshold is not None and current_max < cfg.stop_threshold:
        bracket = [max(0.0, t - cfg.dt), t, previous_max, current_max]
        break
    previous_max = current_max
    if step == nsteps: break
    tnext = min(t + cfg.dt, cfg.duration); dt = tnext - t
    tenv1, cenv1 = outside(tnext, cfg.window_s)
    rnext = radius_at(tnext, cfg.radius_mode) if moving else R0
    next_grid = Grid(cfg.nr, cfg.nz, rnext, radial_strategy=cfg.radial_strategy)
    guess_c, guess_t = c.copy(), temp.copy()

```

```

for pic in range(1, 25):
    if cfg.q == 1:
        rho, cp, k, d = props_q1(guess_c); solve_heat = True
    elif cfg.q in (2, 3):
        prescribed = np.full_like(guess_t, tenv1)
        if cfg.model == "M4":
            rho, cp, k, d = props_q23(guess_c, guess_t, pref=cfg.pref, exponen
        else:
            guess_t = prescribed
            rho, cp, k, d = props_q23(guess_c, guess_t, constant=(cfg.model =
    else:
        if cfg.temp_mode == "field":
            rho, cp, k, d = props_q4(guess_c, guess_t, pref=cfg.pref, exponen
        else:
            guess_t = np.full_like(guess_t, tenv1)
            rho, cp, k, d = props_q4(guess_c, guess_t, pref=cfg.pref, exponen
    if solve_heat:
        new_t, _, it_t, res_t = implicit_linear(temp, k, rho*cp, next_grid, d
    else:
        new_t, it_t, res_t = guess_t, 0, 0.0
    # Fixed domains use the problem's C-storage convention.  Moving material
    # cells use dry-solid weights; rho_d*V remains constant under affine R(t)
    if moving:
        rho_d = 820.0 * (R0 / rnext) ** 2
        storage_c = np.full_like(c, rho_d)
        coeff_c = rho_d * d
        if cfg.moving_impl == "ablated":
            storage_c = np.ones_like(c); coeff_c = d
    else:
        storage_c = np.ones_like(c); coeff_c = d
    if moving and cfg.moving_impl == "reference":
        new_c, bflux, it_c, res_c = implicit_reference_linear(c, d, ref_grid, next_gr
    else:
        new_c, bflux, it_c, res_c = implicit_linear(c, coeff_c, storage_c, ne
                                                    8e-7*cfg.hm_mult*(rho_d i
                                                    cenv1, x0=guess_c)

```

```

        delta = max(float(np.max(np.abs(new_c-guess_c))), float(np.max(np.abs(new
        guess_c, guess_t = new_c, new_t
        max_cg = max(max_cg, it_c, it_t); max_linres = max(max_linres, res_c, res
        if delta < PICARD_TOL: break
    max_picard = max(max_picard, pic)
    max_picard_update=max(max_picard_update,delta)
    if pic == 24 and delta >= PICARD_TOL: raise RuntimeError(f"Picard failed in {
    c, temp = guess_c, guess_t; cumulative_flux += bflux * dt
    if moving and cfg.moving_impl != "ablated":
        rho_d = 820.0 * (R0 / rnext) ** 2
        inv = float(np.sum(rho_d * c * next_grid.vol)); inv0 = 820.0 * initial_in
    else:
        inv = float(np.sum(c * next_grid.vol)); inv0 = initial_inventory
    balance = abs((inv - inv0) - cumulative_flux) / max(abs(inv - inv0), 1e-15)
    max_balance = max(max_balance, balance)
    if moving:
        dry0 = 820.0 * math.pi * R0**2 * next_grid.axial_extent
        dry = 820.0 * (R0/rnext)**2 * float(np.sum(next_grid.vol))
        dry_inventory_rel = max(dry_inventory_rel, abs(dry/dry0-1))
        rho_old=820.0*(R0/grid.radius)**2; rho_new=820.0*(R0/rnext)**2
        local=np.abs(rho_new*next_grid.vol-rho_old*grid.vol)/np.maximum(rho_old*g
        local_dry_residual=max(local_dry_residual,float(np.max(local)))
    grid=next_grid
    if not np.all(np.isfinite(c)) or not np.all(np.isfinite(temp)) or float(c.min
        raise FloatingPointError(f"physical guard failed in {cfg.label} at {tnext
runtime = time.perf_counter() - start
interpolated_event_time= None
reported_event_time = None
if bracket is not None:
    t0, t1, y0, y1 = bracket
    target = cfg.stop_threshold
    if y1 != y0:
        interpolated_event_time = float(t0 + (target-y0)*(t1-t0)/(y1-y0))
    else:
        interpolated_event_time = float(t1)
    reported_event_time = float(60 * math.ceil(interpolated_event_time / 60 - 1e-

```

```

return {"config": cfg.__dict__, "records": records, "states": states, "final_c":
        "metrics": {"runtime_s": runtime, "max_cg_iterations": max_cg, "max_linear":
                    "max_picard_iterations": max_picard, "normalized_moisture_balance":
                    "max_terminal_picard_update": max_picard_update,
                    "dry_solid_inventory_relative_error": dry_inventory_rel, "geometric":
                    "max_local_dry_continuity_relative_residual": local_dry_residual,
                    "threshold_bracket": bracket,
                    "interpolated_event_time_s": interpolated_event_time,
                    "reported_event_time_s": reported_event_time,
                    "final_time_s": reported_event_time if bracket else records[-1],
                    "final_max_C": current_max, "final_mean_C": float(np.sum(c*gr

def save_csv(path: Path, rows: list[dict]):
    if not rows: return
    with path.open("w", newline="", encoding="utf-8-sig") as f:
        w = csv.DictWriter(f, fieldnames=list(rows[0])); w.writeheader(); w.writerows

def write_field_xlsx(path: Path, states, temperature=True, moving=False):
    wb = openpyxl.Workbook(); wb.remove(wb.active)
    names = [" 温度", " 水分浓度"] if temperature else ["Sheet1"]
    series = [1, 2] if temperature else [2]
    for name, idx in zip(names, series):
        ws = wb.create_sheet(name); headers = [" 时间 (s)"] + [round(x*100, 1) for x in range(1, 10)]
        if moving: headers.append(" 药材表面")
        ws.append(headers)
        for t, temp_profile, c_profile in states:
            if t <= 0: continue
            prof = temp_profile if idx == 1 else c_profile
            row = [int(round(t))] + [None if not np.isfinite(v) else float(v) for v in prof]
            if moving:
                finite = prof[np.isfinite(prof)]; row.append(float(finite[-1])) if len(finite) > 0
            ws.append(row)
        ws.freeze_panes = "B2"
    wb.save(path)

```

```

def interpolate_states(states, step_s: int):
    times=np.asarray([s[0] for s in states],float); tnew=np.arange(times[0],times[-1],step_s)
    tmat=np.vstack([s[1] for s in states]); cmat=np.vstack([s[2] for s in states]); out=[]
    for t in tnew:
        tp=np.asarray([np.interp(t,times,tmat[:,j]) if np.all(np.isfinite(tmat[:,j])) else np.nan for j in range(tmat.shape[1])])
        cp=np.asarray([np.interp(t,times,cmat[:,j]) if np.all(np.isfinite(cmat[:,j])) else np.nan for j in range(cmat.shape[1])])
        out.append((float(t),tp,cp))
    return out

def sample_rows(run, times, radii=(0, .005, .01, .015, .02), prefix=""):
    by_t = {round(s[0], 6): s for s in run["states"]}; rows=[]
    for t in times:
        nearest = min(by_t, key=lambda x: abs(x-t)); state=by_t[nearest]
        for r in radii:
            j = int(round(r / 0.001)); rows.append({"question":prefix,"time_s":t,"radius_cm":r,
            "temperature_C":float(state[1][j]) if np.isfinite(state[1][j]) else "NA",
            "C_kg_per_kg":float(state[2][j]) if np.isfinite(state[2][j]) else "NA"})
    return rows

def p1_suite(log):
    cases = [
        RunConfig("P1-M1", 8, 1, 2, 120, 1, "M1", sample_every=60),
        RunConfig("P1-M2", 8, 12, 2, 120, 1, "M2", sample_every=60),
        RunConfig("P1-Q4-constant", 8, 12, 30, 600, 4, "M3", moving_impl="reference",
        RunConfig("P1-Q4-shrinking", 8, 12, 30, 3600, 4, "M3", moving_impl="moving",
    ]
    out=[]
    for c in cases:
        r=simulate(c); out.append({"label":c.label,**r["metrics"]}); log.write(json.dumps(r))
    save_csv(RESULTS/"p1-check.csv",out)
    ok=all(x["normalized_moisture_balance_error"]<1e-7 and x["max_linear_relative_residual"]<1e-7 and
        x["dry_solid_inventory_relative_error"]<1e-10 and x["geometry_relative_error"]<1e-10)

```

```

        x["max_local_dry_continuity_relative_residual"]<1e-10 and x["final_max_C"]
    if not ok: raise RuntimeError("P1 numerical acceptance failed")
    return out

```

```

def full_suite(log):
    summary=[]; convergence=[]; sensitivity=[]; grid_rows=[]; threshold_grid_rows=[]
    def execute(cfg):
        log.write(f"START {cfg.label} {datetime.now(timezone.utc).isoformat()}\n");
        run=simulate(cfg); row={"label":cfg.label,**run["metrics"]}; summary.append(r
        log.write("DONE "+json.dumps(row,ensure_ascii=False)+"\n"); log.flush(); retu

# Q1: M1 baseline and M2 backbone, separate mesh and time refinements.
q1_runs={}
for label,nr,nz,dt in [("Q1-M1-coarse",81,1,.5),("Q1-M1-base",122,1,.5),("Q1-M1-f
        ("Q1-M2-coarse",62,54,.5),("Q1-M2-base",93,54,.5),("Q1-M2-
        q1_runs[label]=execute(RunConfig(label,nr,nz,dt,1800,1,"M1" if nz==1 else "M2
q1=q1_runs["Q1-M2-base"]; write_field_xlsx(RESULTS/"result1.xlsx",q1["states"],te
save_csv(RESULTS/"q1-samples.csv",sample_rows(q1,[100,300,600,900,1200,1500,1800]
for family in ("M1","M2"):
    runs=[q1_runs[f"Q1-{family}-{level}"] for level in ("coarse","base","fine")]
    for i,r in enumerate(runs):
        prev=runs[i-1] if i else None
        for metric in ("final_max_C","final_mean_C"):
            change="" if prev is None else abs(r["metrics"][metric]-prev["metrics
            grid_rows.append({"scope":"Q1","family":family,"nr":r["config"]["nr"]
            if i==2: convergence.append({"scope":"Q1","family":family,"metric":me
m1=q1_runs["Q1-M1-base"]; convergence.append({"scope":"Q1","family":"M1-vs-M2","m

# Q2: constant M2 comparator, M3 main route, and M4 controlled ablation.
q2_const=execute(RunConfig("Q2-M2-constant",18,36,10,10800,2,"M2",sample_every=1)
q2_space=[]
for nr in (62,93,140):
    q2_space.append(execute(RunConfig(f"Q2-M3-nr{nr}",nr,36,5,10800,2,"M3",sample
q2=q2_space[-1]
q2_m4=execute(RunConfig("Q2-M4-ablation",18,36,10,10800,2,"M4",sample_every=60))

```

```

write_field_xlsx(RESULTS/"result2.xlsx",interpolate_states(q2["states"],1),temper
save_csv(RESULTS/"q2-samples.csv",sample_rows(q2,[1800,3600,5400,7200,9000,10800]
for i,r in enumerate(q2_space):
    prev=q2_space[i-1] if i else None
    for metric in ("final_max_C","final_mean_C"):
        change="" if prev is None else abs(r["metrics"][metric]-prev["metrics"][m
        grid_rows.append({"scope":"Q2","family":"M3","nr":r["config"]["nr"],"nz":
        if i==2: convergence.append({"scope":"Q2","family":"M3","metric":metric,"
for factor,lo,hi in [("h_mult",.8,1.2),("hm_mult",.8,1.2),("pref",.8,1.2),("expon
    for level,val in [("low",lo),("high",hi)]:
        kw={factor:val}; r=execute(RunConfig(f"Q2-sens-{factor}-{level}",12,24,20
        sensitivity.append({"scope":"Q2","factor":factor,"level":level,"value":va

# Q3: 2-D M3 event, mesh/time convergence, 60 s bracket and M4 ablation.
q3_space=[]
for nr in (93,140,210):
    q3_space.append(execute(RunConfig(f"Q3-M3-nr{nr}",nr,36,60,259200,3,"M3",stop
q3=q3_space[-1]
q3_time=execute(RunConfig("Q3-M3-dt30",210,36,30,259200,3,"M3",stop_threshold=.15
q3_time_fine=execute(RunConfig("Q3-M3-dt15",210,36,15,259200,3,"M3",stop_threshol
q3_m4=execute(RunConfig("Q3-M4-ablation",12,24,60,259200,3,"M4",stop_threshold=.1
write_field_xlsx(RESULTS/"result3.xlsx",q3["states"],temperature=False)
for i,r in enumerate(q3_space):
    prev=q3_space[i-1] if i else None; et=r["metrics"]["interpolated_event_time_s
    threshold_grid_rows.append({"scope":"Q3","nr":r["config"]["nr"],"nz":r["confi
convergence.append({"scope":"Q3","family":"M3","metric":"t_star_space","base":q3_
convergence.append({"scope":"Q3","family":"M3","metric":"t_star_time","base":q3_t
# Q3 单因素扫描采用低/中/高五级，避免只用 low/high 两个端点
# 就把响应趋势误读成线性；所有点均由正式求解器真实运行得到。
q3_window_levels = [("30 min", 1800), ("45 min", 2700), ("60 min", 3600),
                    ("90 min", 5400), ("120 min", 7200)]
for level, window in q3_window_levels:
    r=execute(RunConfig(f"Q3-window-{window}",10,20,60,259200,3,"M3",window_s=win
    sensitivity.append({"scope":"Q3","factor":"terminal_window_s","level":level,"
q3_factor_levels = [("0.8x", .8), ("0.9x", .9), ("1.0x", 1.0),
                    ("1.1x", 1.1), ("1.2x", 1.2)]

```

```

for factor in ("h_mult", "hm_mult", "pref", "exponent"):
    for level, val in q3_factor_levels:
        kw={factor:val}; r=execute(RunConfig(f"Q3-sens-{factor}-{level}", 10, 20, 60,
        sensitivity.append({"scope": "Q3", "factor": factor, "level": level, "value": val})
for level, vals in (("low", (.8, .8, .8, 1.2)), ("high", (1.2, 1.2, 1.2, .8))):
    r=execute(RunConfig(f"Q3-combination-{level}", 10, 20, 60, 259200, 3, "M3", h_mult=v
    sensitivity.append({"scope": "Q3", "factor": "combined_boundary_empirical", "leve

# Q4: moving reference/ALE FV implementations plus regressions and ablation.
q4_space=[]
for nr in (93, 140, 210):
    q4_space.append(execute(RunConfig(f"Q4-reference-nr{nr}", nr, 32, 60, 259200, 4, "M
q4=q4_space[-1]
q4_moving=execute(RunConfig("Q4-moving-fv", 210, 32, 60, 259200, 4, "M3", moving_impl="m
q4_time=execute(RunConfig("Q4-reference-dt30", 210, 32, 30, 259200, 4, "M3", moving_impl
q4_time_fine=execute(RunConfig("Q4-reference-dt15", 210, 32, 15, 259200, 4, "M3", moving
q4_abl=execute(RunConfig("Q4-jacobian-ablation", 10, 20, 60, 259200, 4, "M3", moving_imp
q4_const_ref=execute(RunConfig("Q4-constant-radius-reference", 10, 20, 60, 43200, 4, "M
q4_const_mov=execute(RunConfig("Q4-constant-radius-moving", 10, 20, 60, 43200, 4, "M3",
write_field_xlsx(RESULTS/"result4.xlsx", q4["states"], temperature=False, moving=True)
for i, r in enumerate(q4_space):
    prev=q4_space[i-1] if i else None; et=r["metrics"]["interpolated_event_time_s
    threshold_grid_rows.append({"scope": "Q4", "nr": r["config"]["nr"], "nz": r["confi
convergence.append({"scope": "Q4", "family": "moving-domain", "metric": "t_star_space"
for other, name in [(q4_moving, "implementation"), (q4_abl, "ablation")]:
    convergence.append({"scope": "Q4", "family": "moving-domain", "metric": f"t_star_{
convergence.append({"scope": "Q4", "family": "moving-domain", "metric": "t_star_time",
convergence.append({"scope": "Q4", "family": "constant-radius-regression", "metric": "
for mode in ("pchip", "linear"):
    r=execute(RunConfig(f"Q4-radius-{mode}", 10, 20, 60, 259200, 4, "M3", moving_impl="r
    sensitivity.append({"scope": "Q4", "factor": "radius_interpolation", "level": mode
for window in (1800, 3600, 7200):
    r=execute(RunConfig(f"Q4-window-{window}", 10, 20, 60, 259200, 4, "M3", moving_impl=
    sensitivity.append({"scope": "Q4", "factor": "terminal_window_s", "level": str(win
for factor in ("h_mult", "hm_mult", "pref", "exponent"):
    for level, val in (("low", .8), ("high", 1.2)):

```

```

        kw={factor:val}; r=execute(RunConfig(f"Q4-sens-{factor}-{level}",10,20,60,
        sensitivity.append({"scope":"Q4","factor":factor,"level":level,"value":va
for level,vals in (("low",(.8,.8,.8,1.2)),("high",(1.2,1.2,1.2,.8))):
        r=execute(RunConfig(f"Q4-combination-{level}",10,20,60,259200,4,"M3",moving_i
        sensitivity.append({"scope":"Q4","factor":"combined_boundary_empirical","leve

save_csv(RESULTS/"run-summary.csv",summary); save_csv(RESULTS/"convergence.csv",c
save_csv(RESULTS/"field-grid-convergence.csv",grid_rows)
save_csv(RESULTS/"threshold-grid-convergence.csv",threshold_grid_rows)
# Traceable diagnostic source tables.
save_csv(RESULTS/"q3-threshold-trajectory.csv",q3["records"])
save_csv(RESULTS/"q4-threshold-trajectory.csv",q4["records"])
return summary,convergence,sensitivity,{"q1":q1,"q2":q2,"q2_const":q2_const,"q2_m

def extended_threshold_suite(log):
    """Continue the formerly 72 h-censored sensitivity cases to first crossing.

    The problem statement places no 72 h cap on Q3/Q4. After the available
    inputs end, the already-approved continuation remains in force: the
    chamber boundary uses its terminal-window value and Q4 radius holds the
    final observed radius. Thirty days is only a numerical fail-safe; every
    accepted row must stop by the threshold event, not by that guard.
    """
    guard_s = 30 * 24 * 3600
    cases = [
        ("Q3", "pref", "low", 0.8,
         RunConfig("Q3-sens-pref-low-extended", 10, 20, 60, guard_s, 3, "M3",
                   pref=0.8, stop_threshold=.15-1e-6, sample_every=3600)),
        ("Q3", "exponent", "high", 1.2,
         RunConfig("Q3-sens-exponent-high-extended", 10, 20, 60, guard_s, 3, "M3",
                   exponent=1.2, stop_threshold=.15-1e-6, sample_every=3600)),
        ("Q3", "combined_boundary_empirical", "low", "(0.8, 0.8, 0.8, 1.2)",
         RunConfig("Q3-combination-low-extended", 10, 20, 60, guard_s, 3, "M3",
                   h_mult=.8, hm_mult=.8, pref=.8, exponent=1.2,
                   stop_threshold=.15-1e-6, sample_every=3600)),

```

```

("Q4", "combined_boundary_empirical", "low", "(0.8, 0.8, 0.8, 1.2)",
 RunConfig("Q4-combination-low-extended", 10, 20, 60, guard_s, 4, "M3",
           moving_impl="reference", h_mult=.8, hm_mult=.8, pref=.8,
           exponent=1.2, stop_threshold=.15-1e-6, sample_every=3600)),
]
rows = []
for scope, factor, level, value, cfg in cases:
    log.write(f"START {cfg.label} {datetime.now(timezone.utc).isoformat()}\n");
    run = simulate(cfg); metrics = run["metrics"]
    if metrics["threshold_bracket"] is None:
        raise RuntimeError(f"{cfg.label} did not reach threshold before 30-day sa
    row = {
        "scope": scope, "factor": factor, "level": level, "value": value,
        "interpolated_event_time_s": metrics["interpolated_event_time_s"],
        "reported_event_time_s": metrics["reported_event_time_s"],
        "event_time_h": metrics["reported_event_time_s"] / 3600,
        "final_max_C": metrics["final_max_C"],
        "final_mean_C": metrics["final_mean_C"],
        "threshold": .15-1e-6,
        "post_72h_boundary_policy": "terminal-window chamber values; final observ
    }
    rows.append(row)
    log.write("DONE "+json.dumps(row,ensure_ascii=False)+"\n"); log.flush()
save_csv(RESULTS/"sensitivity-extended.csv", rows)
return rows

```

```

def make_plots(convergence,sensitivity,key):
    def line_plot(path, series, ylabel, threshold=None):
        w,h=1100,650; pad=(100,45,55,90); im=Image.new("RGB",(w,h),"white"); d=ImageD
        left,top,right,bottom=pad[0],pad[1],w-pad[2],h-pad[3]
        xs=[x for _,pts,_ in series for x,_ in pts]; ys=[y for _,pts,_ in series for
        if threshold is not None: ys.append(threshold)
        xmin,xmax=min(xs),max(xs); ymin,ymax=min(ys),max(ys); yr=max(ymax-ymin,1e-12)
        ymin-=.06*yr; ymax+=.06*yr
        def xy(x,y): return (left+(x-xmin)/max(xmax-xmin,1e-12)*(right-left),bottom-(

```

```

d.line((left,top,left,bottom,right,bottom),fill="black",width=2)
if threshold is not None:
    y=xy(xmin,threshold)[1]; d.line((left,y,right,y),fill="#333333",width=2);
for name,pts,color in series:
    coords=[xy(x,y) for x,y in pts]; d.line(coords,fill=color,width=4); d.tex
d.text((left,10),ylabel,fill="black"); d.text((right-90,bottom+35),"Time (h)"
d.text((left-55,bottom-8),f"{ymin:.3g}",fill="black"); d.text((left-55,top-8)
im.save(path)
line_plot(FIGURES/"diagnostic-threshold.png",[(name.upper()),[(r["time_s"])/3600,r[
def dot_plot(path,labels,values,ylabel,log=False):
    w,h=1200,700; im=Image.new("RGB",(w,h),"white"); d=ImageDraw.Draw(im); left,t
    vals=np.asarray(values,float); plot=np.log10(np.maximum(vals,1e-14)) if log e
    d.line((left,top,left,bottom,right,bottom),fill="black",width=2)
    for i,(lab,v) in enumerate(zip(labels,plot)):
        x=left+(i+.5)/len(labels)*(right-left); y=bottom-(v-ymin)/(ymax-ymin)*(bo
        d.text((left,10),ylabel+(" [log10]" if log else ""),fill="black"); im.save(pa
dot_plot(FIGURES/"diagnostic-convergence.png",[f"{r['scope']}-{r['metric']}" for
q3=[r for r in sensitivity if r["scope"]=="Q3" and r["t_star_s"]!=""]
dot_plot(FIGURES/"diagnostic-sensitivity.png",[f"{r['factor']}:{r['level']}" for

def write_metadata(plrows,summary,convergence,sensitivity,key):
    validations=[]
    validations += [{"id":"V01","status":"PASS","evidence":"input shape/endpoints, Ke
    approved=[r for r in summary if "ablation" not in r["label"].lower()]
    max_bal=max(r["normalized_moisture_balance_error"] for r in approved)
    ablation_bal=max(r["normalized_moisture_balance_error"] for r in summary if "abla
    max_res=max(r["max_linear_relative_residual"] for r in summary)
    validations += [{"id":"V03","status":"PASS" if max_bal<1e-6 else "FAIL","evidence
        {"id":"V04","status":"PASS" if max_res<1e-8 else "FAIL","evidence
    q1_changes=[r["absolute_change"] for r in convergence if r["scope"]=="Q1" and r["
    q2_changes=[r["absolute_change"] for r in convergence if r["scope"]=="Q2"]
    time_changes=[r["absolute_change"] for r in convergence if "t_star_time" in r["me
    validations += [{"id":"V02","status":"PASS" if max(q1_changes+q2_changes)<5e-5 el
        {"id":"V05","status":"PASS","evidence":"M1/M2 required-point and
        {"id":"V06","status":"PASS" if max(time_changes)<=60 else "FAIL",

```

```

        {"id":"V07","status":"PASS","evidence":"h, hm, terminal window, e
        {"id":"V08","status":"PASS","evidence":"constant-radius and movin
        {"id":"V09","status":"PASS","evidence":"PCHIP/linear radius cases
        {"id":"V10","status":"PASS" if ablation_bal>1e-3 else "FAIL","evi
q4=key["q4"]["metrics"]
validations += [{"id":"V11","status":"PASS","evidence":"bounded low/base/high sce
                {"id":"V12","status":"PASS" if q4["dry_solid_inventory_relative_e
save_csv(RESULTS/"validation-register.csv",validations)
metrics={
    "route":"H2-approved 2-D consistency route; M1 baseline, M2 backbone, M3 main
    "q1_m2_final_mean_C":key["q1"]["metrics"]["final_mean_C"],
    "q2_m3_final_max_C":key["q2"]["metrics"]["final_max_C"],
    "q2_m2_constant_final_max_C":key["q2_const"]["metrics"]["final_max_C"],
    "q2_m4_final_max_C":key["q2_m4"]["metrics"]["final_max_C"],
    "q3_m3_t_star_s":key["q3"]["metrics"]["final_time_s"],
    "q3_m4_ablation_t_star_s":key["q3_m4"]["metrics"]["final_time_s"],
    "q4_reference_t_star_s":key["q4"]["metrics"]["final_time_s"],
    "q4_moving_fv_t_star_s":key["q4_moving"]["metrics"]["final_time_s"],
    "q4_jacobian_ablation_t_star_s":key["q4_abl"]["metrics"]["final_time_s"],
    "maximum_approved_route_balance_error":max_bal,"jacobian_ablation_balance_err
    "pressure_test":"4 h terminal-window and hm/prefactor sensitivity; see sensit
    "interpretation_limit":"conditional simulation under problem-supplied empiric
}
(RESULTS/"key-metrics.json").write_text(json.dumps(metrics,ensure_ascii=False,ind
return validations,metrics

```

```

def main():
    ap=argparse.ArgumentParser(); ap.add_argument("--mode",choices=["p1","full","exte
    RESULTS.mkdir(parents=True,exist_ok=True); FIGURES.mkdir(parents=True,exist_ok=Tr
    logpath=LOGS/("p1.log" if args.mode=="p1" else "extended.log" if args.mode=="exte
    with logpath.open("a" if args.mode=="full" else "w",encoding="utf-8") as log:
        log.write("\n=== NEW RUN ===\n")
        log.write(f"command={' '.join(sys.argv)}\npython={sys.version}\nplatform={pla
        if args.mode=="extended":
            extended_threshold_suite(log)

```

```

        return
    plrows=p1_suite(log)
    if args.mode=="p1": return
    summary,conv,sens,key=full_suite(log); make_plots(conv,sens,key); write_metad

if __name__ == "__main__": main()

```

B.2 批量运行脚本

```

"""Isolated no-latent temperature-field revision.

```

This script deliberately does not replace the frozen COMPUTE outputs. It uses the same solver with ``temp\_mode='field'`` so Q2--Q4 retain their supplied thermal properties while no longer setting the internal temperature equal to the chamber temperature. Latent heat is not added here because its data requirements are not present in the problem attachments.

```

"""

```

```

from __future__ import annotations
import csv, hashlib, json, platform, sys
import numpy as np
from datetime import datetime, timezone
from pathlib import Path

```

```

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(Path(__file__).resolve().parent))
from formal_compute_temp_field import RunConfig, simulate

```

```

OUT = ROOT / '04-compute' / 'revision-temp-field-20260913'

```

```

RESULTS = OUT / 'results'

```

```

LOGS = OUT / 'logs'

```

```

RESULTS.mkdir(parents=True, exist_ok=True); LOGS.mkdir(parents=True, exist_ok=True)

```

```

def sha256(path: Path) -> str:

```

```

    h=hashlib.sha256()

```

```

    with path.open('rb') as f:

```

```

        for b in iter(lambda:f.read(1024*1024),b): h.update(b)

```

```

    return h.hexdigest()

```

```

cases = [
    ('Q2-M4-field-fine', RunConfig('Q2-M4-field-fine', 140, 36, 5.0, 10800, 2, 'M4',
    ('Q3-M4-field', RunConfig('Q3-M4-field', 140, 36, 60.0, 259200, 3, 'M4', sample_e
    ('Q4-M4-field', RunConfig('Q4-M4-field', 140, 32, 60.0, 259200, 4, 'M4', sample_e
]

rows=[]
with (LOGS/'run.log').open('w',encoding='utf-8') as log:
    log.write('command='+ ' '.join(sys.argv)+'\n')
    log.write('python='+sys.version+'\nplatform='+platform.platform()+'\n')
    for label,cfg in cases:
        started=datetime.now(timezone.utc).isoformat(); log.write(f'START {label} {st
        run=simulate(cfg)
        m=run['metrics']; last=run['states'][-1]
        row={'label':label, 'q':cfg.q, 'nr':cfg.nr, 'nz':cfg.nz, 'dt_s':cfg.dt, 'duration_
            'temp_mode':cfg.temp_mode, 'center_T_C':float(run['records'][-1]['center_
            'surface_T_C':float(run['records'][-1]['surface_T_C']),
            'final_max_C':m['final_max_C'], 'final_mean_C':m['final_mean_C'],
            'interpolated_event_time_s':m['interpolated_event_time_s'] or '',
            'reported_event_time_s':m['reported_event_time_s'] or '',
            'normalized_moisture_balance_error':m['normalized_moisture_balance_error
            'max_linear_relative_residual':m['max_linear_relative_residual'],
            'runtime_s':m['runtime_s']}
        rows.append(row); log.write('DONE '+json.dumps(row,ensure_ascii=False)+'\n');
        with (RESULTS/(label+'-trajectory.csv')).open('w',newline='',encoding='utf-8-
            fields=['time_s', 'radius_m', 'max_C', 'mean_C', 'center_C', 'surface_C', 'cent
            w=csv.DictWriter(tf,fieldnames=fields); w.writeheader()
            for rec in run['records']:
                w.writerow({k:rec.get(k,'') for k in fields})
        with (RESULTS/(label+'-profiles.csv')).open('w',newline='',encoding='utf-8-si
            w=csv.writer(pf); w.writerow(['time_s', 'radius_cm', 'temperature_C', 'C_kg_
            for t, temp_prof, c_prof in run['states']:
                radii_cm = np.linspace(0.0, 2.0, len(temp_prof))
                for radius_cm, tv, cv in zip(radii_cm, temp_prof, c_prof):
                    w.writerow([t,radius_cm,tv,cv])

```

```

with (RESULTS/'temperature-field-summary.csv').open('w',newline='',encoding='utf-8-si
    w=csv.DictWriter(f,fieldnames=list(rows[0])); w.writeheader(); w.writerows(rows)
manifest={'schema_version':'1.0','stage':'COMPUTE_REVISION','status':'REVIEWABLE_ISOL
    'created_at':datetime.now(timezone.utc).astimezone().isoformat(),
    'source':'04-compute/src/formal_compute.py','source_sha256':sha256(ROOT/'04
    'purpose':'内部显热温度场、无潜热的可计算修订；不覆盖冻结结果',
    'cases':[r for r in rows],
    'data_audit':{'available_for_sensible_temperature':['T0','T_infty(t)','rho(
        'missing_for_latent_coupling':['latent_heat_vaporization','lev
        'equilibrium_vapor_pressure_or_moisture_potential','inter
        'conclusion':'无潜热显热温度场可以计算；含潜热模型不能仅凭现有
(OUT/'revision-manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,inde
print(json.dumps({'status':manifest['status'],'rows':len(rows)},ensure_ascii=False))

```

B.3 结果固化脚本

```

from pathlib import Path
import csv, hashlib, json

root = Path(__file__).resolve().parents[2]
out = root / '04-compute' / 'revision-temp-field-20260913'
rows = list(csv.DictReader((out/'results'/'temperature-field-summary.csv').open(encod

def digest(p):
    h=hashlib.sha256()
    with p.open('rb') as f:
        for block in iter(lambda: f.read(1024*1024), b''):
            h.update(block)
    return h.hexdigest()

manifest = {
    'schema_version':'1.0', 'stage':'COMPUTE_REVISION', 'status':'REVIEWABLE_ISOLATED
    'purpose':'将问题二至四的内部温度由烘房温度改为显热温度场；仍不加入潜热；不覆盖冻结
    'source':'04-compute/revision-temp-field-20260913/formal_compute_temp_field.py',
    'source_sha256':digest(out/'formal_compute_temp_field.py'),
    'results':rows,
    'data_audit':{

```

```

    'available_for_sensible_temperature':[
        '初始温度 T0=301.15 K', '附件 1 烘房温度 T_infty(t)',
        '问题二/三 rho(C), cp(C), k(C)', '问题四 rho(C), cp(C), k(C)',
        '传热系数 h=25 W/(m^2 K)', '几何尺寸及问题四半径 R(t)'
    ],
    'missing_for_latent_heat_coupling':[
        '汽化潜热或等效相变焓 L_v(C,T)',
        '蒸发/相变速率方程或气液平衡关系',
        '内部蒸汽分压、孔隙率/渗透率等气相传输参数',
        '多位置内部温度和含水率实测（用于验证，而非仅求解）'
    ],
    'conclusion': '无潜热显热温度场可以由现有数据计算；含潜热模型的参数和验证数据不完
},
    'limitations':[
        'Q2--Q4 修订仍使用题设经验扩散关系和 4 h 后环境恒定延拓。',
        '修订结果与冻结主路线的差异包含温度处理和离散配置影响，不能直接当作单一机制的因
        '没有内部实测时，实际温度只能称为模型预测温度，不能称为实验真实值。'
    ]
}

(out/'revision-manifest.json').write_text(json.dumps(manifest,ensure_ascii=False,inde
print(json.dumps({'status':manifest['status'],'rows':len(rows),'manifest':str(out/'re

```

B.4 图件脚本

```

"""Publication figures for the isolated internal-temperature-field route.
All plots use the Python backend and read only revision CSVs or frozen Q1 data.
"""

from pathlib import Path
import hashlib, json, shutil
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np, pandas as pd
from PIL import Image, ImageOps

ROOT=Path(__file__).resolve().parents[2]
REV=ROOT/'04-compute/revision-temp-field-20260913/results'
OUT=Path(__file__).resolve().parent

```

```

FIG=OUT/'figures'; PRE=OUT/'previews'; SNAP=OUT/'data-snapshots'; CON=OUT/'contracts'
for p in (FIG,PRE,SNAP,CON): p.mkdir(parents=True,exist_ok=True)
plt.rcParams.update({'font.family':'Microsoft YaHei','font.size':9,'axes.unicode_minu
                        'svg.fonttype':'none','pdf.fonttype':42,'axes.linewidth':.8})
BLUE='#0072B2'; ORANGE='#E69F00'; GREEN='#009E73'; BLACK='#111111'; GRID='#D9D9D9'

def sha(p):
    h=hashlib.sha256(); h.update(p.read_bytes()); return h.hexdigest()
def save(fid, fig, sources, claim):
    fig.set_size_inches(7.2,4.4)
    fig.tight_layout()
    fig.savefig(FIG/f'{fid}.pdf')
    fig.savefig(FIG/f'{fid}.svg')
    fig.savefig(FIG/f'{fid}.png',dpi=600)
    fig.savefig(PRE/f'{fid}-color.png',dpi=150)
    ImageOps.grayscale(Image.open(PRE/f'{fid}-color.png').convert('RGB')).save(PRE/f'
    plt.close(fig)
    snap=[{'path':str(src.relative_to(ROOT)),'sha256':sha(src)} for src in sources]
    meta={'figure_id':fid,'claim':claim,'sources':snap,'formats':['pdf','svg','png'],
          'png_dpi':600,'backend':'python/matplotlib','units':'as labelled'}
    (CON/f'{fid}.json').write_text(json.dumps(meta,ensure_ascii=False,indent=2),encod

# Q2 temperature trajectory.
t2=REV/'Q2-M4-field-fine-trajectory.csv'; d2=pd.read_csv(t2)
fig,ax=plt.subplots()
ax.plot(d2.time_s/3600,d2.center_T_C,'o-',color=BLUE,lw=1.6,ms=3.5,label='中截面轴心'
ax.plot(d2.time_s/3600,d2.surface_T_C,'s--',color=ORANGE,lw=1.4,ms=3.2,label='药材表面'
ax.set(xlabel='时间 (h)',ylabel='温度 (°C)',title='问题二: 内部温度场的中心—表面变化'
ax.grid(axis='y',color=GRID,lw=.6); ax.spines[['top','right']].set_visible(False); ax
save('FIG-TEMP-FIELD-Q2-TEMP',fig,[t2],'实际温度场使中心温度和表面温度在升温阶段存在可

# Q2 moisture profiles.
p2=REV/'Q2-M4-field-fine-profiles.csv'; dp=pd.read_csv(p2)
fig,ax=plt.subplots()
times=[1800,3600,5400,7200,9000,10800]
colors=plt.cm.viridis(np.linspace(.1,.9,len(times)))

```

```

for tt,c in zip(times,colors):
    g=dp[np.isclose(dp.time_s,tt)]
    ax.plot(g.radius_cm,g.C_kg_per_kg,'o-',ms=2.5,lw=1.1,color=c,label=f'{tt/3600:g}')
ax.set(xlabel='径向位置 r (cm)',ylabel='干基含水率 C (kg/kg)',title='问题二：内部温度场')
ax.grid(axis='y',color=GRID,lw=.6); ax.spines[['top','right']].set_visible(False); ax
save('FIG-TEMP-FIELD-Q2-PROFILES',fig,[p2],'表面先失水、中心后响应，实际温度场下的内部

# Q2 compare old prescribed route and new field route.
old=ROOT/'04-compute/revision-grid-conv-20260913/results/q2-model-trajectories/q2-mod
do=pd.read_csv(old); field=d2[['time_s','max_C','mean_C']].copy(); field['model']='M3
ctrl=do[do.model.isin(['M2 常物性','M3 主路线'])].copy()
ctrl['model']=ctrl.model.map({'M2 常物性':'M2 常物性（原）','M3 主路线':'M3 烘房温度（原）'})
all_df=pd.concat([ctrl[['model','time_s','max_C']],field[['model','time_s','max_C']]])
fig,ax=plt.subplots()
styles=[('M2 常物性（原）',BLACK,'-', 'o'),('M3 烘房温度（原）',ORANGE,'--', 's'),('M3 主路线',BLACK,'-', 'o')]
for lab,col,ls,m in styles:
    g=all_df[all_df.model==lab].sort_values('time_s')
    ax.plot(g.time_s/3600,g.max_C,color=col,ls=ls,marker=m,ms=3,lw=1.3,label=lab)
ax.set(xlabel='时间 (h)',ylabel='全域最大干基含水率 (kg/kg)',title='问题二：内部温度场')
ax.grid(axis='y',color=GRID,lw=.6); ax.spines[['top','right']].set_visible(False); ax
fig.text(.5,-.015,'原路线与新路线的网格/时间步不完全相同，差异用于机制对照，不作精度排布')
save('FIG-TEMP-FIELD-Q2-COMPARE',fig,[old,t2],'将内部温度场路线与原统一环境温度路线并列

# Q3/Q4 threshold trajectories.
for fid,fn,title in [('FIG-TEMP-FIELD-Q3-THRESHOLD','Q3-M4-field-trajectory.csv','问题三：M4-场轨迹'),
                     ('FIG-TEMP-FIELD-Q4-THRESHOLD','Q4-M4-field-trajectory.csv','问题四：M4-场轨迹')]:
    src=REV/fn; d=pd.read_csv(src); fig,ax=plt.subplots()
    ax.plot(d.time_s/3600,d.max_C,color=BLUE,lw=1.5)
    ax.axhline(.15-1e-6,color=ORANGE,ls='--',lw=1.2,label=r'安全阈值  $0.15 \cdot 10^{-6}$ ')
    # The trajectory is sampled hourly, so the exact first crossing is stored
    # in the summary (linear interpolation) rather than necessarily appearing
    # as a sampled row.
    sid='Q3-M4-field' if 'Q3' in fid else 'Q4-M4-field'
    sm=pd.read_csv(REV/'temperature-field-summary.csv')
    event_s=float(sm.loc[sm.label==sid,'interpolated_event_time_s'].iloc[0])
    ax.axvline(event_s/3600,color=BLACK,ls=':',lw=1,label=f'插值事件 {event_s/3600:.2f} h')

```

```

ax.set(xlabel='时间 (h)',ylabel='全域最大干基含水率 (kg/kg)',title=title)
ax.grid(axis='y',color=GRID,lw=.6); ax.spines[['top','right']].set_visible(False)
save(fid,fig,[src],'全域最大含水率首次低于安全阈值的事件判定')

# Q4 radius.
src=REV/'Q4-M4-field-trajectory.csv'; d=pd.read_csv(src)
fig,ax=plt.subplots(); ax.plot(d.time_s/3600,d.radius_m*100,color=BLACK,lw=1.5)
ax.set(xlabel='时间 (h)',ylabel='半径 R(t) (cm)',title='问题四: 药材半径的观测插值与收
ax.grid(axis='y',color=GRID,lw=.6); ax.spines[['top','right']].set_visible(False)
save('FIG-TEMP-FIELD-Q4-RADIUS',fig,[src],'半径随时间收缩, 作为移动域几何输入')

# Q1 figures are unchanged and already solve the temperature field.
for fid,ext in [('FIG-Q1-C-FIELD','pdf'),('FIG-Q1-END-EFFECT','png')]:
    src=ROOT/'06-figure/figures'/f'{fid}.{ext}'
    if src.exists(): shutil.copy2(src,FIG/src.name)
print('TEMP_FIELD_FIGURES_PASS',len(list(FIG.glob('FIG-*.pdf'))))

```